Palm OS Programming Recipes

Recipe: Using Extended Gadgets

If you want to define a custom user interface object—a UI object outside the standard UI objects defined by Palm OS® software—then you will need to use a gadget. Generally, a gadget defines an area of the screen in which you will manage the user's interaction by watching all tap events. For Palm OS 3.5 and later, you can declare a gadget handler callback function that makes using gadgets easier.

NOTE: If you need to support applications that run on versions of the Palm OS earlier than Palm OS 3.5, then read the information on using gadgets prior to Palm OS 3.5. The information below describes how to use an extended gadget for applications on Palm OS 3.5 and later.

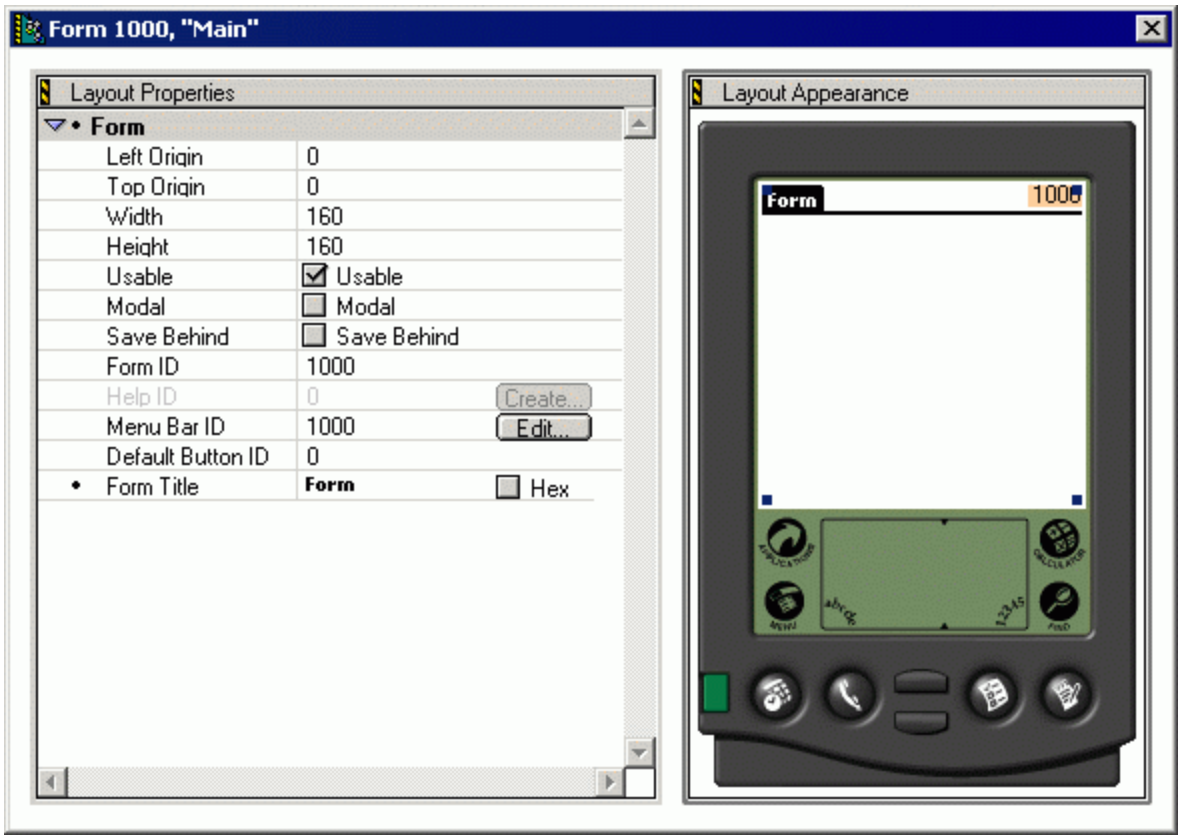
To use an extended gadget in your application, you need to do the following:

- Use Constructor for Palm OS to add the gadget to the user interface.
- Add code to your application to manage the gadget:
  1. Define the minimum OS version as Palm OS 3.5.
  2. Code the gadget handler callback function.
  3. Add the gadget handler callback function to your application's `AppHandleEvent` hander.
- Understand how to access the gadget structure within the gadget hander.
- Optionally send events directly to the callback function.
- Optionally define a basic gadget to support OS versions prior to Palm OS 3.5.
- Understand compatibility issues regarding extended gadgets.

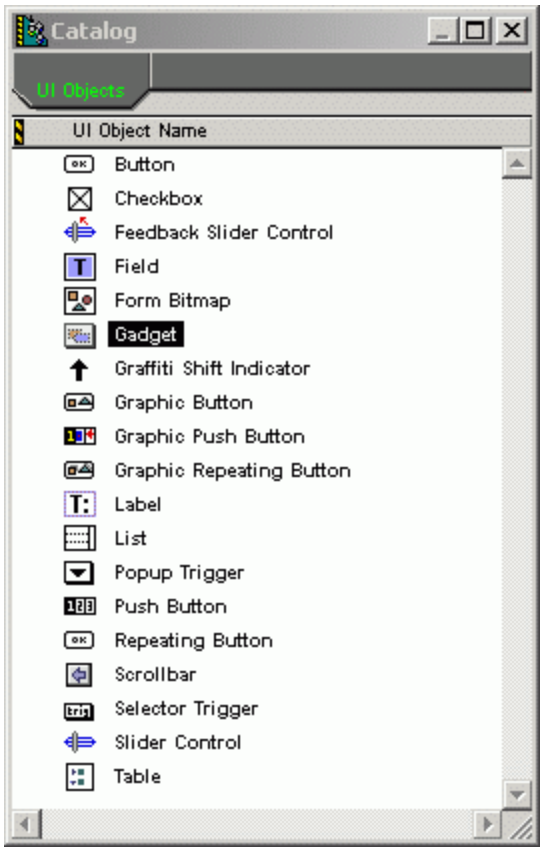
Add the Gadget to the User Interface

Use Constructor for Palm OS to add the gadget to your application's user interface.

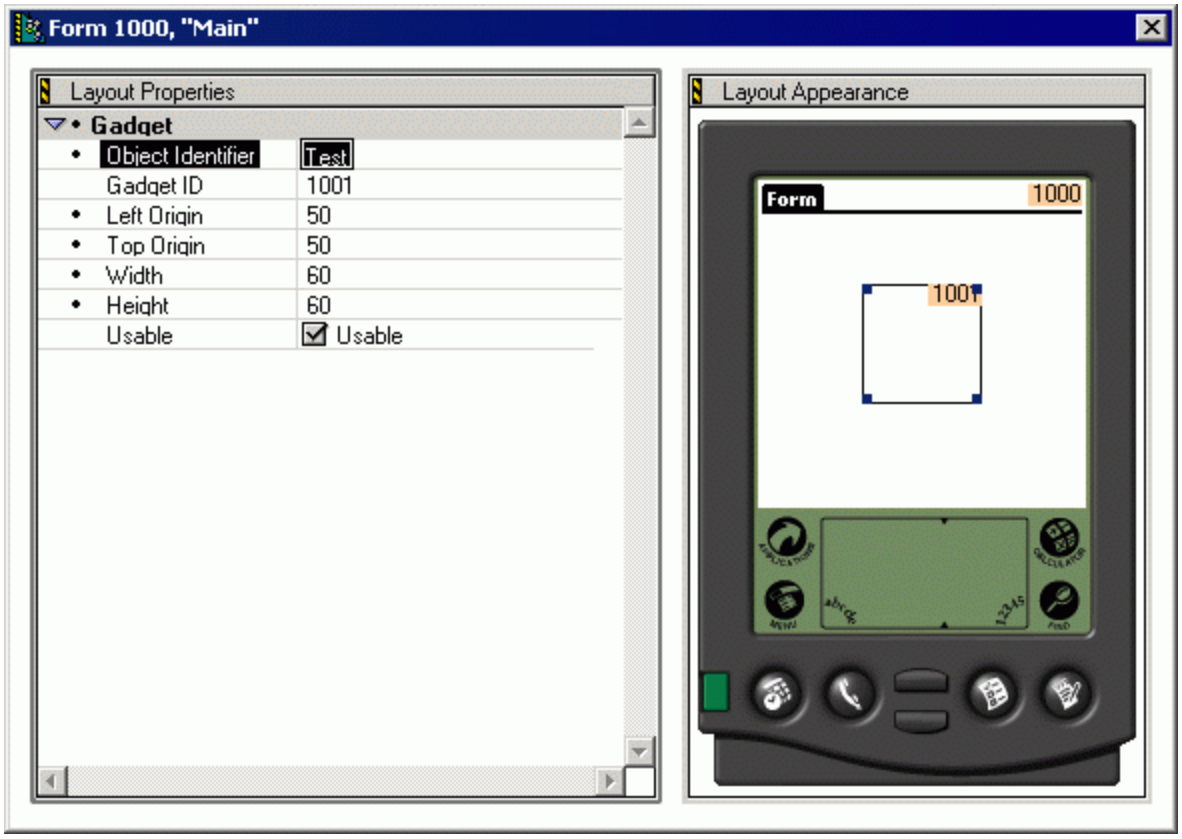
1. Open your application's form in the Form Layout window.



2. Select the Gadget control in the Catalog window.



3. Drag and drop the Gadget control onto the form in the Form Layout window.



4. Update the Gadget control's properties, setting the **Object Identifier**, **Left Origin**, **Top Origin**, **Width**, and **Height**, as appropriate. Leave **Usable** checked. **Define the Minimum OS Version**

If you use the default stationery for CodeWarrior, there is a constant located toward the top of the code called `kOurMinVersion`. Update this constant to indicate Palm OS 3.5:

```
#define kOurMinVersion sysMakeROMVersion(3,5,0,sysROMStageRelease,0)
```

Define the Gadget Handler Callback Function

You can use the following code as an example of a gadget callback function. It acts as a handler for the gadget, taking care of the following gadget actions:

- Drawing and redrawing the gadget
- Handling pen events and form events
- Erasing the gadget
- Deleting the gadget

```
static Boolean TestGadgetHandler(FormGadgetTypeInCallback* pGadget,
       UInt16 cmd, void* pParam)
{
  FormType* frmP;
  Boolean handled;
```

```
//   the gadget handler is called with one of 4 cmd:
//   formGadgetDrawCmd to draw and initialize the gadget
//   formGadgetHandleEventCmd in response to a gadget tap or an
//   event
//     keyed by your application
//   formGadgetEraseCmd if there is any custom screen
//   erasing that needs
//     to be done when it is erased
//   formGadgetDeletionCmd to clean up any memory
//   used for the gadget

////////////////////////////////
//use formGadgetDrawCmd as a chance to initialize, draw, or
//redraw your gadget
////////////////////////////////
switch (cmd)
{
case formGadgetDrawCmd:
   //initialize the gadget data or redraw the gadget
   if (pGadget->data == NULL)
   {
          pGadget->data = (void*)
           MemPtrNew(sizeof(GadgetDataType));
          ErrFatalDisplayIf(theError,
           "Could Not allocate gadget data");
   }
    WinDrawRectangleFrame(rectangleFrame,
     &(pGadget->rect));
   handled = true;
   break;

   ////////////////////////////////
   //cmd is formGadgetHandleEventCmd when the gadget
   //is asked to handle an
   //event that has come in to it.Either it is a
   //response to a tap in the
   //gadgets bounds, or the program has sent something
   //over to be handled.
   //We can tell which one of these it is by looking at
   //the eType of pParam.
   ////////////////////////////////

case formGadgetHandleEventCmd:
   //you need to cast pParam to an EventType* to use it
   EventType* pToEvent = (EventType*) pParam;

   //pToEvent points to the event we need to handle.
   //if it is eType
   //frmGadgetEnterEvent then there was a tap in the
   //gadget's bounds
```

```
    if (pToEvent->eType == frmGadgetEnterEvent)
    {
       //you may find it useful to track if they
       //lift the pen still
       //within the boundaries of the gadget
       Boolean isPenDown = true;
       Int16 newPointX, newPointY;
       UInt16 ticksPerSec = SysTicksPerSecond();

       //track the pen down event
       while (isPenDown){
          EvtGetPen(&newPointX, &newPointY,
            &isPenDown);

          //it is a good idea to put a delay such
          //that the system does
          //not need to fetch pen events more often
          //then need be
          SysTaskDelay(ticksPerSec / 10);
       }

       if (RctPtInRectangle (newPointX, newPointY,
             &(pGadget->rect)));
       {
          //the pen up was also in the gadget
       }

    }
    else if (pToEvent->eType == frmGadgetMiscEvent)
    {
       //you can send events into the gadget
       //handler here
       if (pToEvent->data.gadgetMisc.selector ==
             gadgetDoSomethingButtonSelector){
          //do something here
               }
    }

    handled = true;
    break;

  ////////////////////////////////
  //This is sent to erase the gadget. Generally,
  //FrmHideObject takes care of
  //hiding the object, so return false.
  ////////////////////////////////
  case formGadgetEraseCmd:
     //erase the frame drawn around the gadget
     WinEraseRectangleFrame (rectangleFrame,
```

```
      &(pGadget->rect));
    handled = false;
    break;

  ////////////////////////////////
  //When the gadget is being deleted... So clean up any
  //memory allocated.
  ////////////////////////////////
  case formGadgetDeleteCmd:
    //free the chunk set to hold the gadget's data
    MemPtrFree((MemPtr)pGadget->data);
    handled = true;
    break;

  default:
    break;

  }//Switch(cmd)

  return handled;
}
```

Add the Gadget Handler Callback Function to Your Application

Use `FrmSetGadgetHandler` to set the handler callback function for your gadget. Place the call to `FrmSetGadgetHandler` right after you specify the form handler in `AppHandleEvent`.

```
case MainForm:
  FrmSetEventHandler(frmP, MainFormHandleEvent);

  //SET THE EVENT HANDLER FOR THE GADGET
  FrmSetGadgetHandler(frmP,
            FrmGetObjectIndex(frmP, MainTestGadget),
            TestGadgetHandler);
  break;
```

Accessing the Gadget Structure

Accessing the gadget structure within the gadget handler is acceptable and will be supported in future versions of the Palm OS. Depending on the version of Palm OS Emulator that you are using, you may see warning messages about direct access to system structures when you try to use the gadget structure. Download version 3.3 or later of Emulator, and you will not see this warning message.

The above code does depend on compilation using atleast the update 1 to the OS 4 SDK. Prior to the update the Gadget Handler's first parameter was of type FormGadgetType which is an opaque type. In the update it was switched to `FormGadgetTypeInCallback` which is not opaque.

Send Events to the Gadget Handler Callback Function

You may find it useful to send an event of type `frmGadgetMiscEvent` to the gadget handler callback function. Sending this event will cause Palm OS to call your callback function with a `formGadgetHandleEventCmd`, where `pParam` points to the event you put in the queue.

This procedure can be useful if, for example, you have buttons or other controls that effect the gadget, such as a button that tells the gadget to clear or change the data stored in the gadget. You can thus force the gadget handler to take care of any gadget related action by adding a `frmGadgetMiscEvent` into the queue by calling `EvtAddEventToQueue`.

```
EventType gadgetEvent;

gadgetEvent.data.gadgetMisc.gadgetID = MainTestGadget;
gadgetEvent.data.gadgetMisc.gadgetP = (FormGadgetType*)
      GetObjectPtr(MainTestGadget);
//given that gadgetDoSomethingButtonSelector is defined
//as some number in the header file
gadgetEvent.data.gadgetMisc.selector =
      gadgetDoSomethingButtonSelector;

//this is a frmGadgetMiscEvent
gadgetEvent.eType = frmGadgetMiscEvent;

EvtAddEventToQueue(&gadgetEvent);
```

Support OS Versions Prior to Palm OS 3.5

For backwards compatibility, you may find it useful to code both the basic gadget and an extended gadget callback handler. To support earlier versions of Palm OS, code your application to check the OS version first. Then use the extended gadget callback handler only if the OS version is 3.5 or later.

Understanding Compatibility Issues

When using basic gadgets prior to Palm OS 3.5, you could overlap other UI objects with a gadget, if the overlap did not inhibit taps on the non-gadget UI object. However, with Palm OS 3.5 and later, a tap on a gadget generates an event of type `frmGadgetEnterEvent`. As a result, overlapping a UI control with a gadget will lead to unpredictable results and is not recommended.